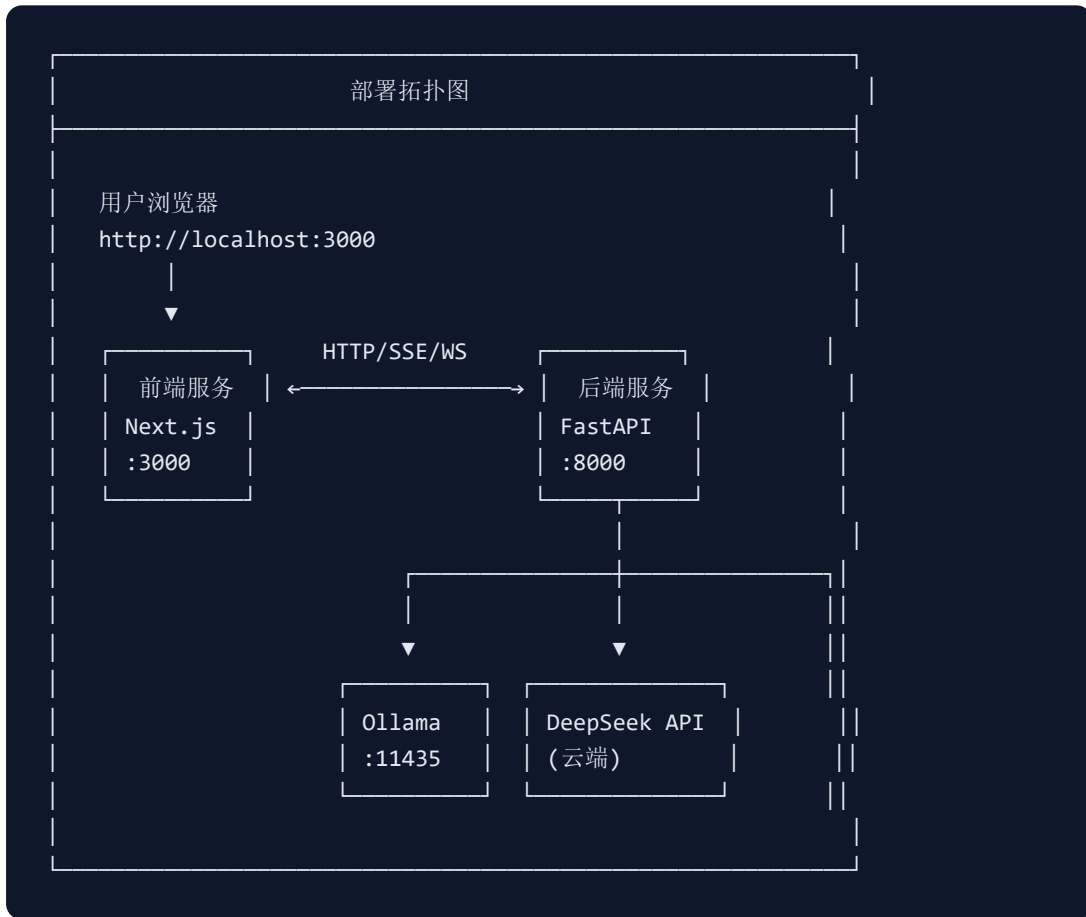


# AgentMatrix 部署文档

多智能体动态协同与国产算力优化平台  
版本: v0.1.0 | 日期: 2026-05-17

## 一、部署架构概览



## 二、环境要求

2.1 硬件要求

| 环境       | 最低配置                     | 推荐配置                            |
|----------|--------------------------|---------------------------------|
| 开发环境     | CPU 4核, 内存 8GB, 磁盘 20GB  | CPU 8核, 内存 16GB, 磁盘 50GB        |
| 生产环境     | CPU 4核, 内存 16GB, 磁盘 50GB | CPU 8核, 内存 32GB, 磁盘 100GB       |
| GPU (可选) | -                        | NVIDIA GPU 8GB+ VRAM (加速本地模型推理) |

2.2 软件要求

| 软件      | 最低版本    | 用途       |
|---------|---------|----------|
| Python  | 3.12+   | 后端运行环境   |
| Node.js | 20.0+   | 前端运行环境   |
| Ollama  | 0.23.0+ | 本地模型推理服务 |
| Git     | 2.40+   | 代码版本管理   |

2.3 操作系统支持

| 操作系统          | 支持状态   | 备注              |
|---------------|--------|-----------------|
| Windows 10/11 | ✅ 完全支持 | PowerShell 5.1+ |
| macOS 12+     | ✅ 完全支持 | 原生终端            |
| Ubuntu 22.04+ | ✅ 完全支持 | Bash            |

三、本地模型部署 (Ollama)

3.1 安装 Ollama

## Windows:

```
# 1. 下载安装包
# https://ollama.com/download/windows

# 2. 安装完成后验证
ollama --version
```

## Linux:

```
curl -fsSL https://ollama.com/install.sh | sh
```

## macOS:

```
brew install ollama
```

## 3.2 配置 Ollama 端口

默认情况下，Ollama 监听 11434 端口。如果该端口被占用，可以通过环境变量修改：

```
# Linux/macOS
export OLLAMA_HOST="0.0.0.0:11435"
ollama serve

# Windows PowerShell
$env:OLLAMA_HOST="0.0.0.0:11435"
ollama serve
```

本项目的默认配置使用 11435 端口：

```
# backend/.env
OLLAMA_HOST=http://localhost:11435
```

## 3.3 下载本地模型

```
# 下载 Qwen2.5 1.5B 模型 (~986MB)
ollama pull qwen2.5:1.5b

# 下载 Phi-4 Mini 3.8B 模型 (~2.5GB)
ollama pull phi4-mini:3.8b
```

```
# 验证模型已安装
ollama list
```

预期输出:

| NAME           | ID           | SIZE   | MODIFIED |
|----------------|--------------|--------|----------|
| phi4-mini:3.8b | 78fad5d182a7 | 2.5 GB | ...      |
| qwen2.5:1.5b   | 65ec06548149 | 986 MB | ...      |

### 3.4 验证 Ollama 服务

```
# 测试 Ollama 是否正常运行
curl http://localhost:11435/api/tags

# 预期返回:
# {"models":[{"name":"qwen2.5:1.5b",...}, {"name":"phi4-mini:3.8b",...}]}
```

## 四、后端服务部署

### 4.1 获取代码

```
git clone <repository-url>
cd AgentMatrix
```

### 4.2 安装后端依赖

方式一：使用 pip 直接安装

```
cd backend

# 创建虚拟环境（推荐）
python -m venv venv

# 激活虚拟环境
# Windows:
venv\Scripts\activate
# Linux/macOS:
source venv/bin/activate
```

```
# 安装依赖
pip install -e .
```

## 方式二：使用 requirements 手动安装

```
cd backend
pip install fastapi>=0.110.0 uvicorn>=0.29.0 pydantic>=2.6.0 \
python-dotenv>=1.0.0 requests>=2.31.0 aiohttp>=3.9.0 \
sqlalchemy>=2.0.0 python-pptx>=0.6.23 python-docx>=1.1.0 \
pymdown-extensions>=10.0.0 ollama>=0.1.0 \
google-generativeai>=0.5.0 numpy>=1.26.0 scipy>=1.12.0 \
loguru>=0.7.0 websockets>=12.0 psutil>=5.9.0
```

## 4.3 配置环境变量

创建 `backend/.env` 文件：

```
# 服务配置
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=true

# 日志配置
LOG_LEVEL=INFO
LOG_FILE=logs/system.log

# 数据库配置
DATABASE_URL=sqlite:///./agentmatrix.db

# Ollama 配置
OLLAMA_HOST=http://localhost:11435
OLLAMA_MODEL=qwen2.5:1.5b
OLLAMA_REVIEW_MODEL=phi4-mini:3.8b

# DeepSeek API 配置（可选，不配置则仅使用本地模型）
DEEPSEEK_API_KEY=your_deepseek_api_key_here
DEEPSEEK_API_BASE=https://api.deepseek.com/v1
DEEPSEEK_MODEL=deepseek-r1-distill

# 复杂度阈值（0-1，超过此值调用云端）
COMPLEXITY_THRESHOLD=0.65

# CORS 配置
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000
```

## 4.4 初始化数据库

```
cd backend

# 首次启动会自动创建数据库表
python -c "from app.database import init_db; import asyncio; asyncio.run(init_db())"
```

## 4.5 启动后端服务

```
cd backend

# 开发模式（支持热重载）
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# 生产模式（多 worker）
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

启动成功日志：

```
INFO:      Started reloader process [xxxxx] using StatReload
INFO:      Database tables created successfully
INFO:      Database initialized successfully
INFO:      Loaded knowledge base with 20 keywords
INFO:      All agents initialized successfully
INFO:      WebSocket manager initialized
INFO:      Application startup complete.
```

## 4.6 验证后端服务

```
# 健康检查
curl http://localhost:8000/health

# 预期返回：
# {"status":"healthy","agents":{"..."},"version":"0.1.0"}

# 查看 API 文档
# 浏览器访问：http://localhost:8000/docs
```

## 五、前端服务部署

---

### 5.1 安装前端依赖

```
cd frontend
npm install
```

### 5.2 配置环境变量

创建 `frontend/.env.local` 文件：

```
# API 地址
NEXT_PUBLIC_API_URL=http://localhost:8000

# WebSocket 地址
NEXT_PUBLIC_WS_URL=ws://localhost:8000
```

### 5.3 启动前端服务

```
cd frontend

# 开发模式
npm run dev

# 生产构建
npm run build
npm start
```

启动成功日志：

```
▲ Next.js 14.1.0
- Local:      http://localhost:3000
- Environments: .env.local, .env
✓ Ready in 5.4s
```

### 5.4 验证前端服务

浏览器访问 `http://localhost:3000`，应能看到 AgentMatrix 主界面。

---

## 六、一键启动脚本

---

### 6.1 Windows PowerShell 脚本

项目提供了 `start.ps1` 一键启动脚本：

```
# 在项目根目录运行
.\start.ps1
```

脚本会自动完成： 1. 检查 Python 和 Node.js 环境 2. 检查后端依赖安装情况 3. 检查前端依赖安装情况 4. 启动后端服务 (:8000) 5. 启动前端服务 (:3000)

### 6.2 Windows 批处理脚本

```
# 在项目根目录运行
.\start.bat
```

## 七、生产环境部署

---

### 7.1 使用 systemd (Linux)

后端服务：

```
# /etc/systemd/system/agentmatrix-backend.service
[Unit]
Description=AgentMatrix Backend Service
After=network.target ollama.service

[Service]
Type=simple
User=agentmatrix
WorkingDirectory=/opt/agentmatrix/backend
Environment=PATH=/opt/agentmatrix/backend/venv/bin:/usr/bin
ExecStart=/opt/agentmatrix/backend/venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
Restart=always
RestartSec=5
```



```
[Install]
WantedBy=multi-user.target
```

### 前端服务:

```
# /etc/systemd/system/agentmatrix-frontend.service
[Unit]
Description=AgentMatrix Frontend Service
After=network.target agentmatrix-backend.service

[Service]
Type=simple
User=agentmatrix
WorkingDirectory=/opt/agentmatrix/frontend
ExecStart=/usr/bin/npm start
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
# 启动服务
sudo systemctl daemon-reload
sudo systemctl enable agentmatrix-backend agentmatrix-frontend
sudo systemctl start agentmatrix-backend agentmatrix-frontend

# 查看状态
sudo systemctl status agentmatrix-backend
sudo systemctl status agentmatrix-frontend
```

## 7.2 使用 Docker (推荐)

### Dockerfile - 后端:

```
# backend/Dockerfile
FROM python:3.12-slim

WORKDIR /app
COPY . .
RUN pip install -e .

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

### Dockerfile - 前端:

```
# frontend/Dockerfile
FROM node:20-alpine

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

EXPOSE 3000
CMD ["npm", "start"]
```

### **docker-compose.yml:**

```
version: '3.8'

services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      - OLLAMA_HOST=http://ollama:11434
      - DEEPSEEK_API_KEY=${DEEPSEEK_API_KEY}
    volumes:
      - ./backend/data:/app/data
    depends_on:
      - ollama
    restart: unless-stopped

  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    environment:
      - NEXT_PUBLIC_API_URL=http://backend:8000
      - NEXT_PUBLIC_WS_URL=ws://backend:8000
    depends_on:
      - backend
    restart: unless-stopped

  ollama:
    image: ollama/ollama:latest
    ports:
      - "11435:11434"
    volumes:
      - ollama_data:/root/.ollama
    restart: unless-stopped
```

```
volumes:
  ollama_data:
```

```
# 启动全部服务
docker-compose up -d

# 下载模型到容器
docker-compose exec ollama ollama pull qwen2.5:1.5b
docker-compose exec ollama ollama pull phi4-mini:3.8b
```

---

## 八、Nginx 反向代理配置（生产环境）

---

```
server {
    listen 80;
    server_name agentmatrix.example.com;

    # 前端
    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # 后端 API
    location /api/ {
        proxy_pass http://localhost:8000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    # WebSocket
    location /ws {
        proxy_pass http://localhost:8000/ws;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
    }
}
```

## 九、监控与健康检查

### 9.1 健康检查端点

| 端点                           | 方法  | 用途                  |
|------------------------------|-----|---------------------|
| /health                      | GET | 系统整体健康状态 + Agent 状态 |
| /api/v1/metrics              | GET | 系统运行指标              |
| /api/v1/chat/health          | GET | 聊天服务状态              |
| /api/v1/workflow/cache/stats | GET | 工作流缓存统计             |

### 9.2 日志配置

```
# backend/.env
LOG_LEVEL=INFO          # 开发: DEBUG, 生产: INFO 或 WARNING
LOG_FILE=logs/system.log
```

日志文件位置: backend/logs/system.log

### 9.3 性能监控

系统指标 ( GET /api/v1/metrics ) 包含: - total\_requests - 总请求数 - local\_executions - 本地执行次数 - cloud\_executions - 云端执行次数 - api\_calls - API 调用次数 - cost\_saved - 节省费用估算

## 十、常见问题排查

### 10.1 Ollama 连接失败

现象: 后端日志 "Failed to call Ollama: Connection refused"  
解决:

1. 检查 Ollama 是否在运行: `ollama list`
2. 检查端口是否正确: 默认 11434, 本项目配置 11435
3. 检查环境变量: `echo $env:OLLAMA_HOST` (Windows)
4. 手动测试连接: `curl http://localhost:11435/api/tags`

## 10.2 端口被占用

现象: uvicorn 启动报 "Address already in use"

解决:

1. 查找占用进程: `netstat -ano | findstr :8000`
2. 修改端口: 在 `.env` 中设置 `SERVER_PORT=8001`

## 10.3 前端 API 请求 404

现象: 浏览器控制台显示 `/api/v1/...` 返回 404

解决:

1. 检查 `.env.local` 中 `NEXT_PUBLIC_API_URL` 是否正确
2. 确认后端服务正在运行
3. 检查后端是否在正确的端口上运行

## 10.4 DeepSeek API 调用失败

现象: 返回 "Error: DeepSeek API Key 未设置"

解决:

1. 检查 `backend/.env` 中 `DEEPSEEK_API_KEY` 是否已设置
2. 确认 API Key 是否有效 (未过期)
3. 可以在前端系统设置中配置 API Key

# 十一、安全注意事项

1. **API 密钥管理:** 不要将 `.env` 文件提交到版本控制。已加入 `.gitignore`。
2. **生产环境 CORS:** 修改 `ALLOWED_ORIGINS` 为实际前端域名, 不要使用 `*`。
3. **防火墙:** 生产环境应配置防火墙, 仅开放必要端口 (80/443)。
4. **HTTPS:** 生产环境应使用 SSL/TLS 加密通信。

5. **定期更新:** 定期更新 Ollama 模型和后端依赖, 获取安全补丁。